

AI REAL ESTATE AGENT

LLM Prompt Chaining, Supervised ML, Prompt Engineering & Containerized Deployment

■ **DEADLINE** — Thursday midnight

THE PITCH

You are building an AI real estate agent. A user describes a property in plain English — your system extracts the details, predicts a price using a trained ML model, and explains the result in context. A working LLM prompt chain: natural language in, structured intelligence out, served from a Docker container. *"How much would a 3-bedroom ranch with a big garage in a good neighborhood cost?"* — Your agent parses that into features, predicts a price, and tells the user why.

ARCHITECTURE — PROMPT CHAIN

This project is a **two-stage LLM prompt chain** connected by an ML prediction. Stage 1's output feeds the model. The model's output feeds Stage 2. Each stage has its own prompt, its own Pydantic schema, and its own job.

Query → LLM Stage 1 (extract features → Pydantic) → ML predict → LLM Stage 2 (interpret) → Response → FastAPI (Docker) → UI

DATASET

Ames Housing (recommended) — 2,930 sales, 80+ features. Pick **8–12 key features** (e.g. sqft, beds, baths, neighborhood, quality, garage, year built). Target: SalePrice. Source: [kaggle.com/c/house-prices-advanced-regression-techniques](https://www.kaggle.com/c/house-prices-advanced-regression-techniques)

Bring your own — 500+ rows, one categorical column, one with missing values, meaningful target.

WHAT YOU NEED TO BUILD

— ML Pipeline —

01 Split the Dataset

Three-way split: train / validation / test. No leakage. Ever.

02 Handle Missing Values

Audit nulls on training set. Justify every decision. Fit on train only.

03 Encode and Scale

Distinguish ordinal from nominal. All transformers fit on train only.

04 scikit-learn Pipeline

ColumnTransformer + at least two swappable model types. Serialize your best model.

05 Compare and Report

Train vs validation scores. Evaluate best model on test exactly once.

— LLM Prompt Chain —

06 Stage 1 — Feature Extraction

The LLM parses a natural language query into typed feature values matching your Pydantic schema. Include a completeness signal — which features were confidently extracted vs. which are missing. Do not silently fill gaps with defaults. If the user says "3-bed in a good area" and your model needs 12 features, Stage 1 reports what it found and what is still needed. The UI lets the user review and fill gaps before prediction runs.

07 Stage 2 — Prediction Interpretation

Feed Stage 2 the extracted features, the prediction, and summary stats from your training data (median price, typical range). The interpretation goes beyond the number — is it high or low, what is driving it, how does it compare. This is the second link in the chain: Stage 1 output → ML → Stage 2 prompt.

08 Prompt Versioning

Two prompt variants for extraction. Run both on 3+ test queries. Log: version, input, output, validation result. Pick the winner with evidence. Prompts are code.

09 Schemas and Error Handling

Two Pydantic schemas minimum: extracted features (with completeness metadata) and combined response. Catch API errors, validation failures, malformed output. Return a fallback. Demonstrate one failure case.

— Deployment —

10 FastAPI + Docker

One POST route: query in, validated JSON out. Model loads at startup. Write a Dockerfile from scratch — image includes trained model, dependencies, and app code. Build, run, map the port. If it is not accessible from outside the container, it is not done.

11 UI

Streamlit, Gradio, or HTML. Shows what the LLM extracted, lets the user review or fill missing features, displays the prediction and interpretation. Handles errors gracefully.

THINK ABOUT

How does Stage 1's output become Stage 2's input — what connects the chain?

What should the UI show when the LLM only extracts 4 out of 12 features?

What makes an interpretation useful vs. a restated number?

How does your trained model get from a notebook into a Docker container?

How do you know your pipeline has no data leakage?

Your problems. No hints.

★ BONUS

Add **market insights** as a second capability. Classify user intent — prediction or analysis? For insight queries, feed pre-computed stats to the LLM and narrate real findings. Every number traces to your data.

DELIVERABLES

- Colab notebook — EDA, ML pipeline, prompt experiments (Anyone with link → Viewer)
- GitHub repo — FastAPI app, Dockerfile, serialized model, schemas
- Friday presentation — live demo with insights on your approach, challenges, and design decisions

Project 2 - [Your Name] | Dataset: [name] | Best model: [name] | Test RMSE/Accuracy: [number]

LLM: [name] | Prompt versions: [number] | Docker image: [name:tag] | Notebook: [link] | Repo: [link]

RULES

- **NO VIBE CODING** — Understand every line. You will be asked.
- **NO COPY-PASTE DOCKERFILES** — Explain every instruction or you do not understand it.

Ship it. Thursday midnight. No excuses.